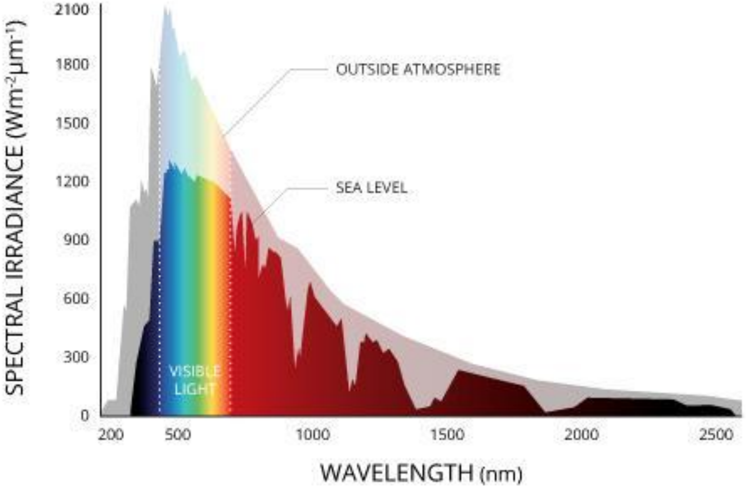
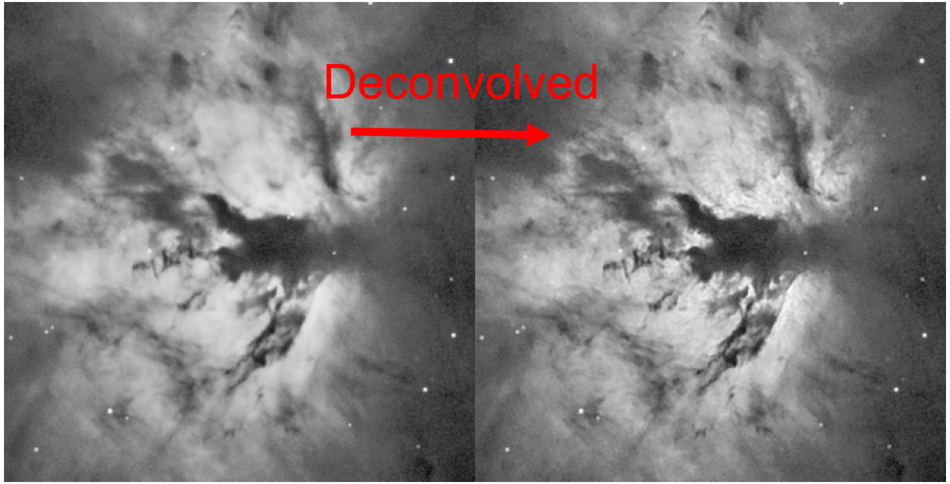
Signal Processing Techniques in the Frequency Domain

IC/DAQ Day 5

Overview

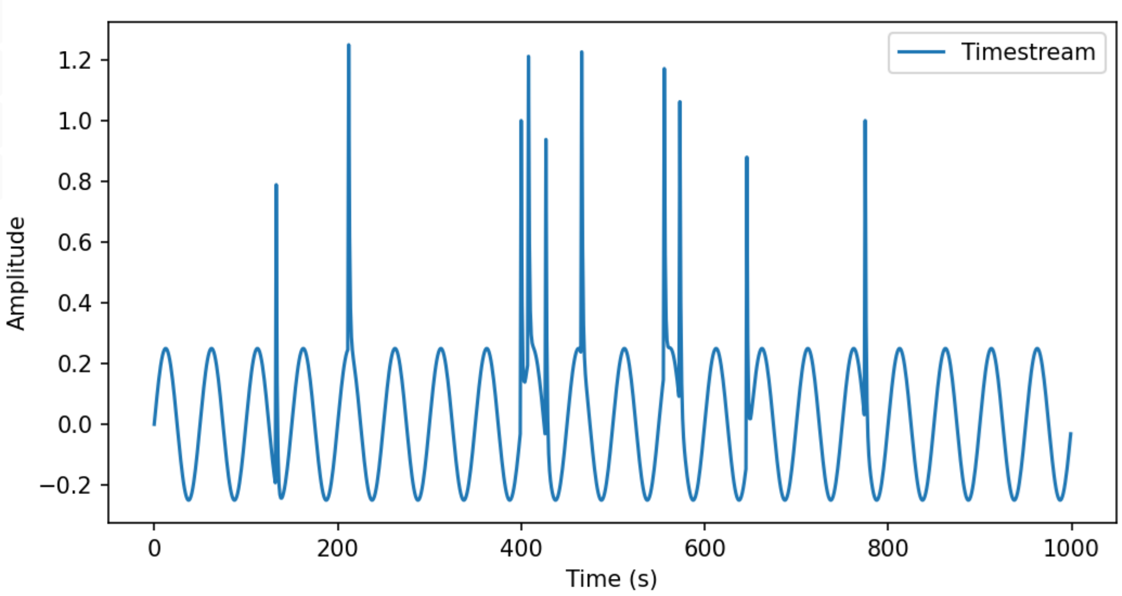
- The Fourier transform lets us move between the time domain and frequency domain.
- Why do we want to work in the frequency domain?
  - Easy to isolate and remove noise and unwanted signals → **filtering, deconvolution**
  - Contains important astronomical information → **spectroscopy/power spectrums, interferometry**





Low pass filtering

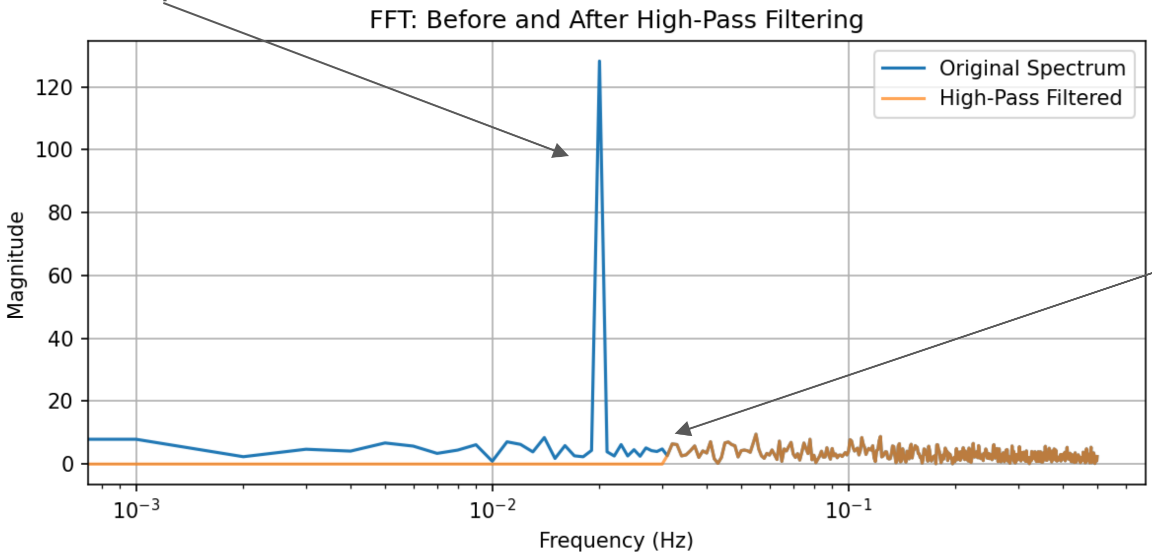
- Everyone has to deal with noisy signals.
- Consider this signal with some pulses on a slowly changing background



Low pass filtering

- Apply FFT and the ripple is condensed to one point.
- The most basic low pass filter is to define a hard cutoff frequency.

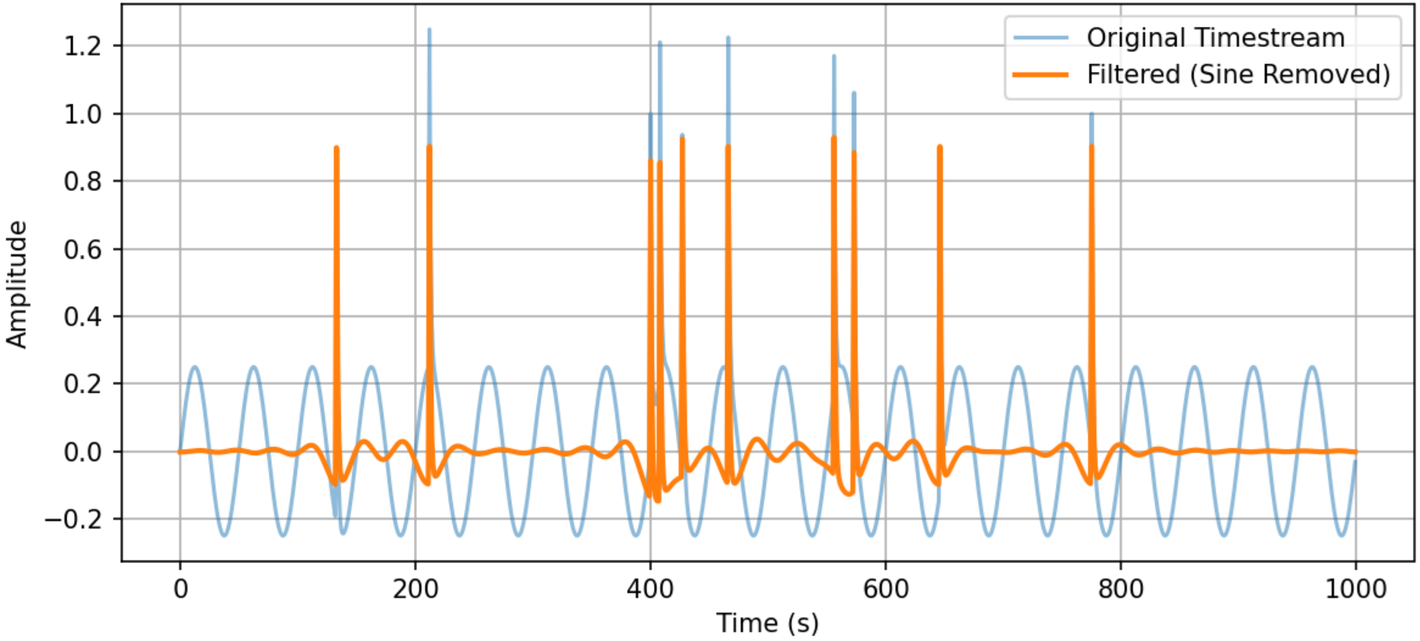
Sine freq



Set the value to 0 for frequencies below the cutoff

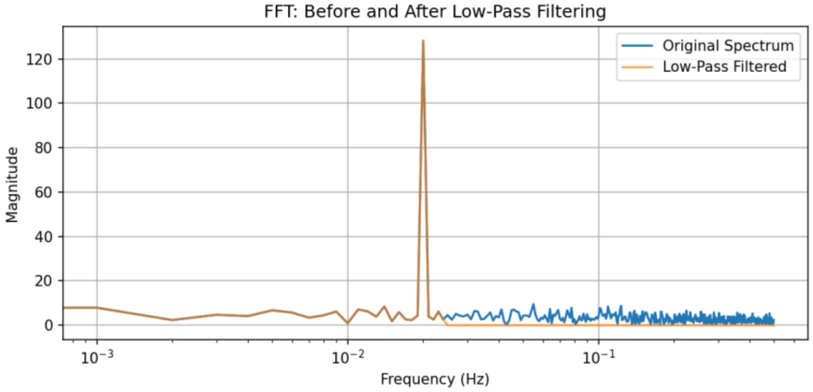
Low pass filtering

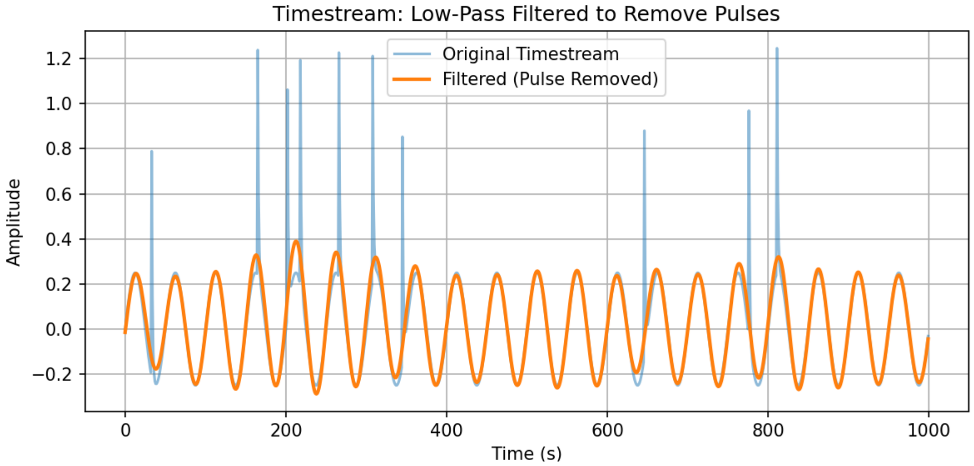
- Transform back with an inverse Fourier transform
- Timestream is now much cleaner because of the flat baseline



High(LOW) pass filtering

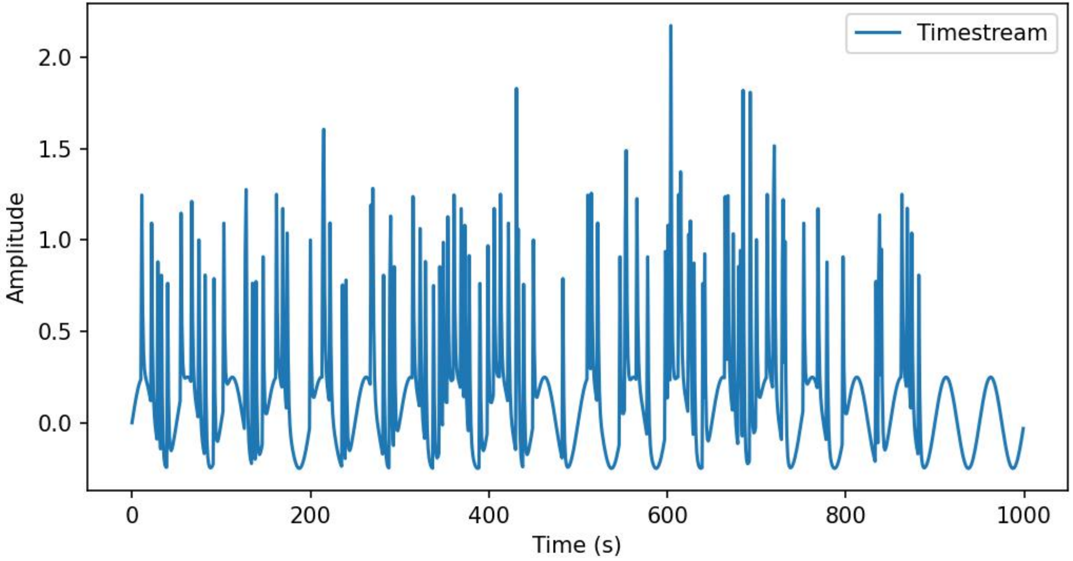
- Suppose now that we want to measure the sine and ignore the pulses.
- Exact same idea, except now we remove the high frequency component in the frequency domain.





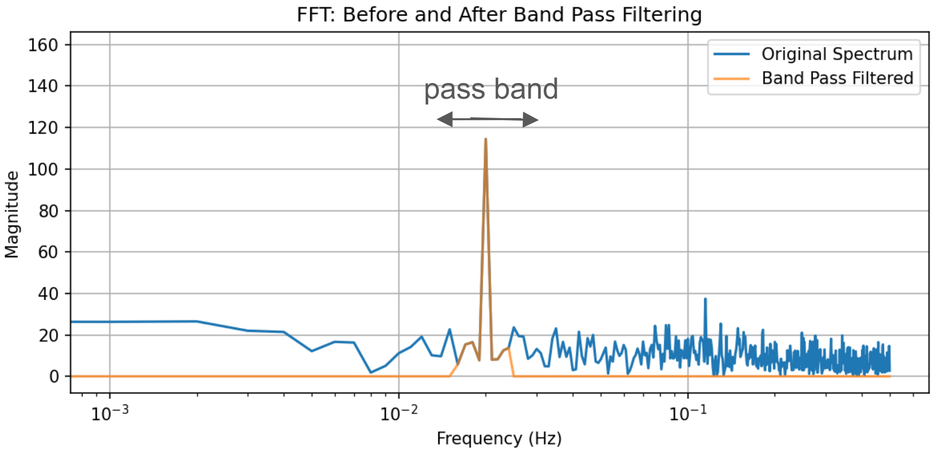
Band pass filtering

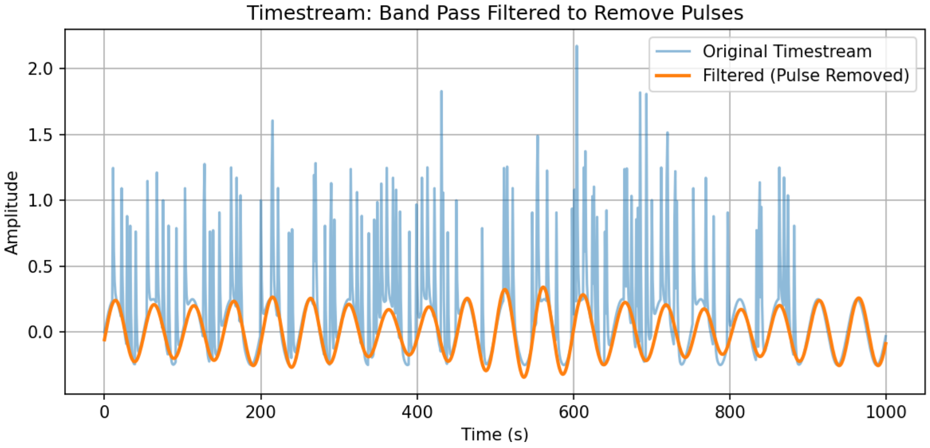
- We can do even better by only looking at a band [f_low, f_high] that contains the peak we see in the frequency domain.
- Suppose you had many more pulses to remove:



Band pass filtering

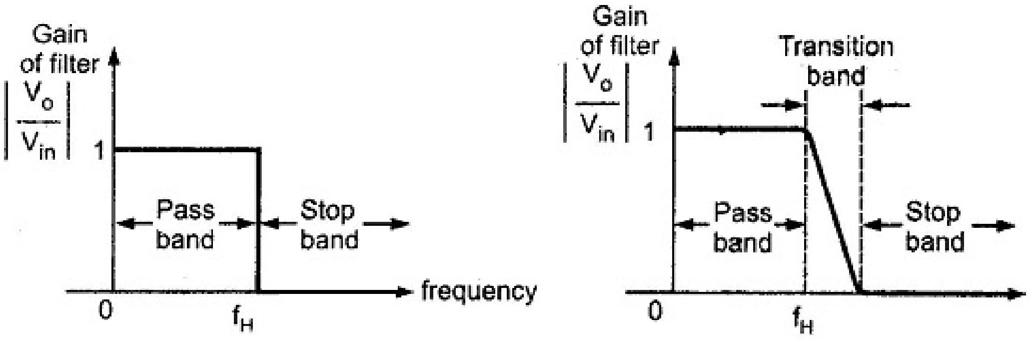
- Separating the sine from the pulses can be used to either get the sine wave background or the pulse timestream by doing (timestream - background)





Types of Low Pass, High Pass, Band Pass Filters

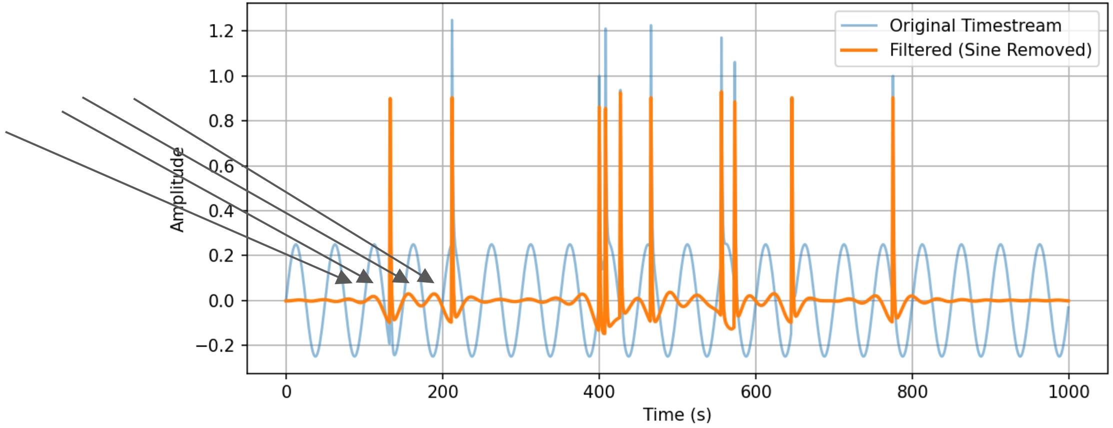
- There are many different types of filtering algorithms that can all be considered a low, high, or band pass filter.
- In general, the different types of filters are defined by how they transition between the frequency range they allow signal (“Pass band”) and reject signal (“Stop band”)



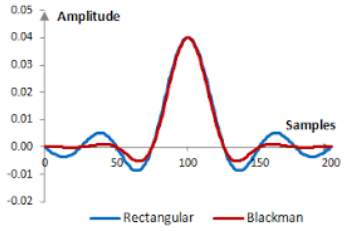
(a) Ideal Fig. 2.71 Low pass filter (b) Practical

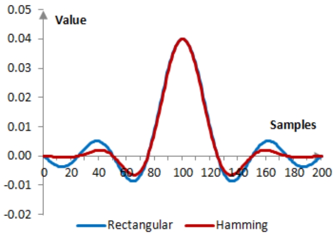
When is an “ideal” filter not ideal?

- Can introduce ripples:



- Examples of smoother filtering methods:
  - Chebyshev filter - Good because sharp transitions, bad because ripples in the the passband
  - Butterworth filter - Good because flat bass band, bad because slower rolloff/transition
  - Blackman windowing and Hamming windowing - Ripples are more dampened compared to rectangular (ideal) filter





Convolutions

- Recall that a **convolution** of functions *f* and *g* is defined as

$$f * g \equiv \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

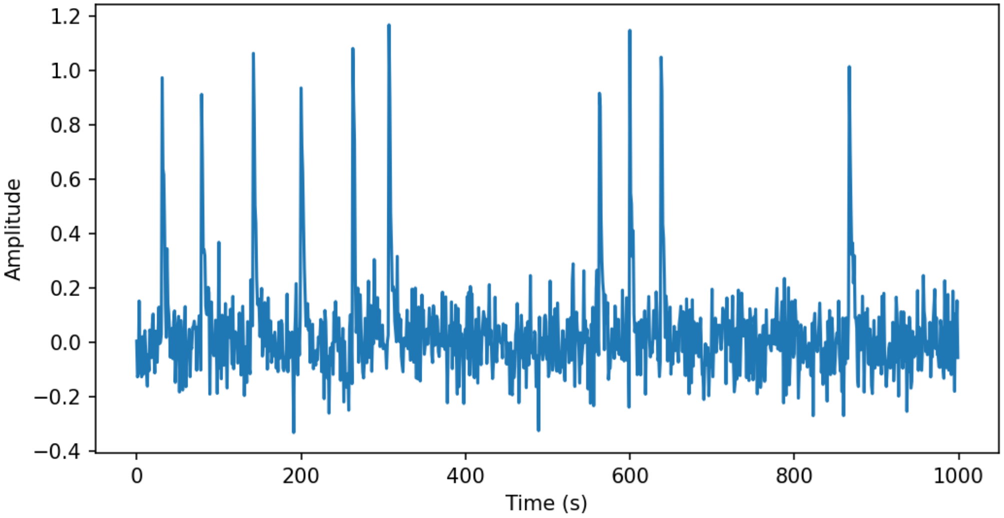
- **Convolution Theorem**: Let *F(w)* and *G(w)* be the convolutions of *f(t)* and *g(t).* Then the fourier transform of the convolution is the product of *F* and *G.*

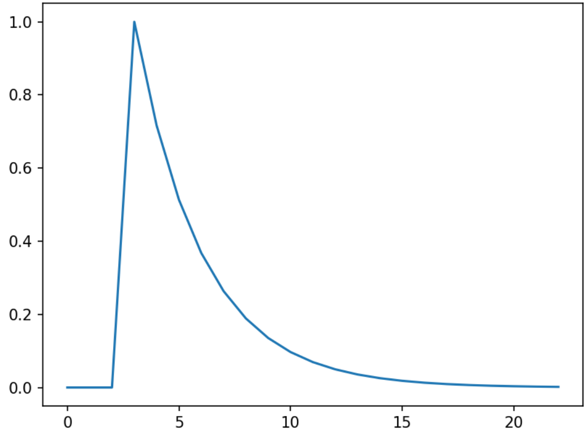
  FFT{f∗g}(w) = F(w)G(w)

  Thus, fourier transforming a convolution of time domain functions is the same as multiplication in the frequency domain.

Matched Filtering

- If you have a signal that you know the shape of, you can use convolutions to increase the signal to noise ratio.
- Very useful to astronomy/physics because we usually have a theoretical model for the type of signal we are looking for (e.g. particle detection, gravitational wave detection).





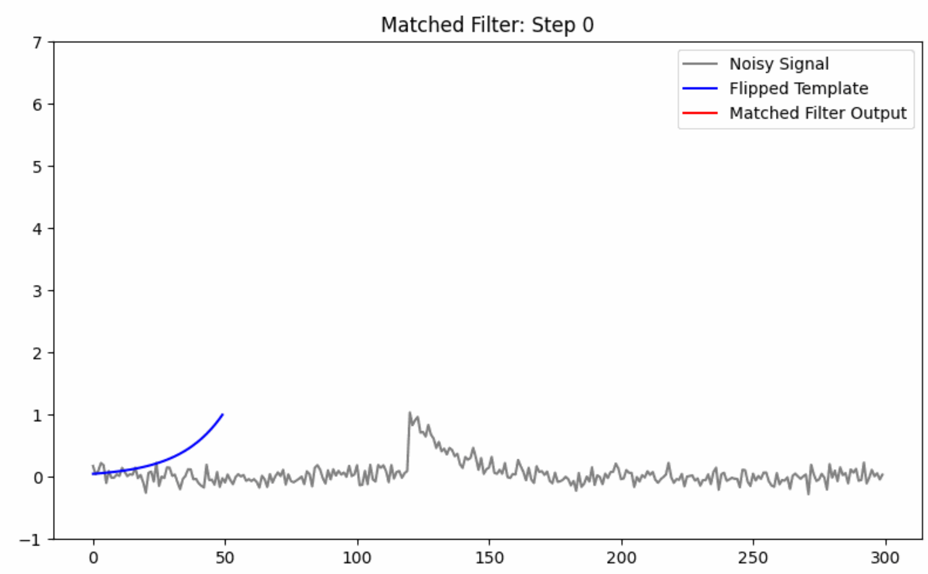
Matched Filtering

- Why does this work?
  - The convolution enhances the parts of the signal where there there is a lot of overlap with the template (i.e. that part of the signal resembles the template) while suppressing the unrelated noise.

$$f * g \equiv \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

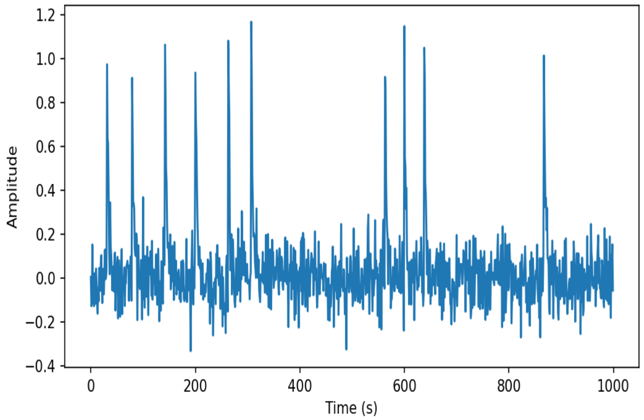
We can understand this equation visually by seeing that

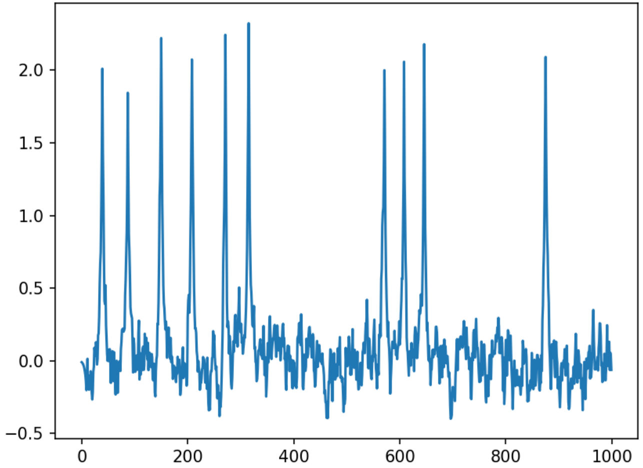
- f*g is large when the area under both curves has a large multiplication product
- f*g is small when when the area under both curves has a small multiplication product



Matched Filtering

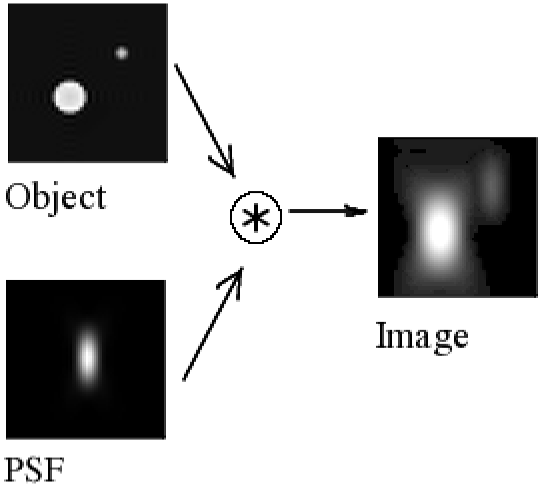
- Looking at the filtered data (signal convolved with template), we can see that the signal to noise ratio as increased.
- Although it is easy to see by eye in this example, matched filtering is a powerful technique when you want to write code that detects events.
- **- also lose time resolution**

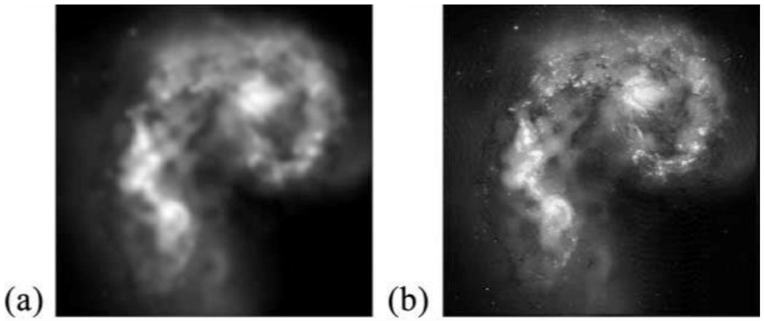




Deconvolution

- The **transfer function** of a measurement system mathematically describes how every input will be transformed at the output of the system.
- For example, the point spread function (PSF) of a telescope tells how each point of light that goes into the telescope will appear to an observer.





Deconvolution

- Suppose you are making an observation with a telescope. The signal you receive will have the true astronomical data mixed with the transfer function (e.g. PSF) of the telescope, plus some random noise (N).

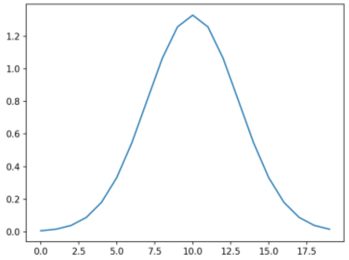
  $$\text{Observation} = \text{True Signal} * \text{PSF} + \text{N}$$

  This “mixing” of the signal and PSF is done via convolution ($*$).

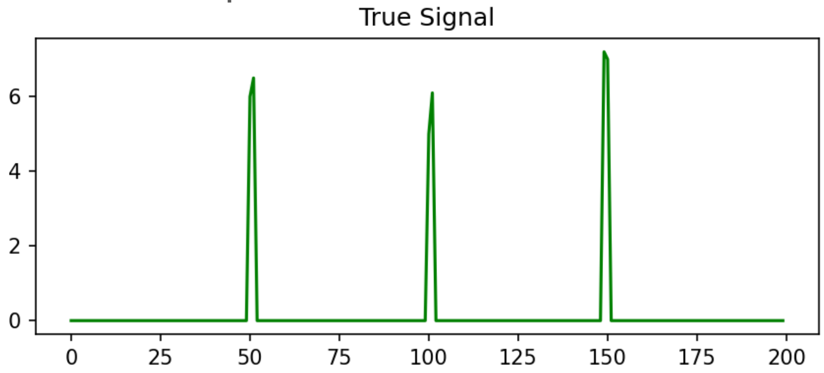
- If the transfer function of a system can be measured by itself, we can **deconvolve** the observed signal to recover the true signal.

Deconvolution Example

- We want to take a blurry signal and make it sharper with deconvolution.
- Suppose we are trying to measure a spectra, but our spectrometer has low resolution. We measure the Gaussian PSF of our system before taking any observations:

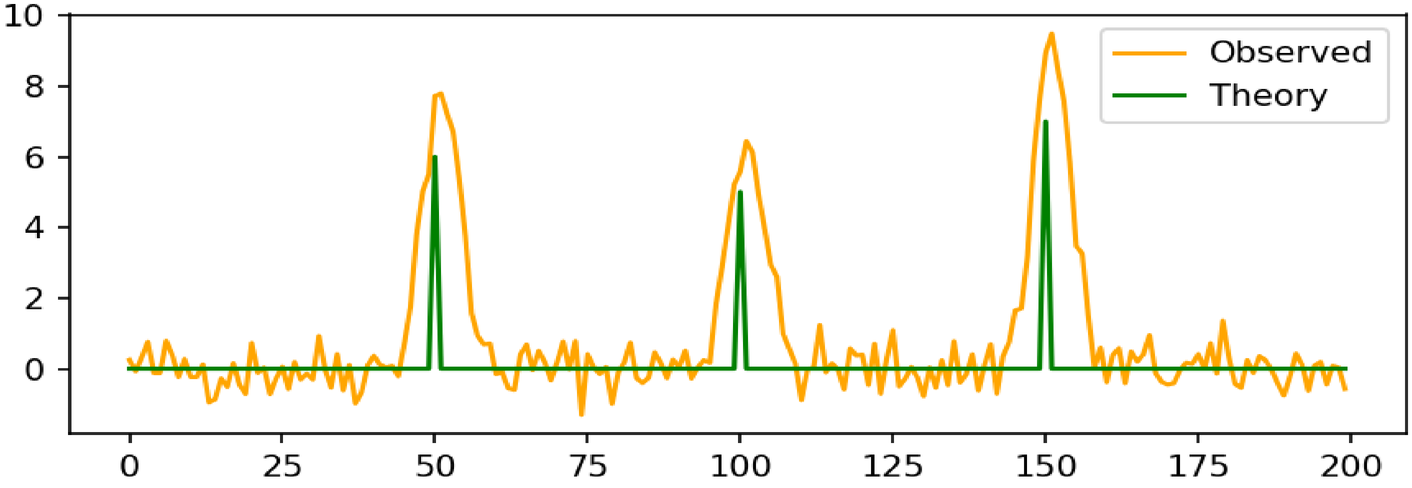


- We want to look for these three spectral lines:



Deconvolution Example

- We make this observation and see that the desired peaks are present, but they are too wide to compare to the exact expected spectral line values.



Deconvolution Example

- Using the Richardson-Lucy deconvolution algorithm, we can undo the blurring of the point spread function.
- The recovered signal is much sharper than the original observation. We can now precisely compare the theory and observation.